

Raster-Vector Interactions

Alexander Abuabara

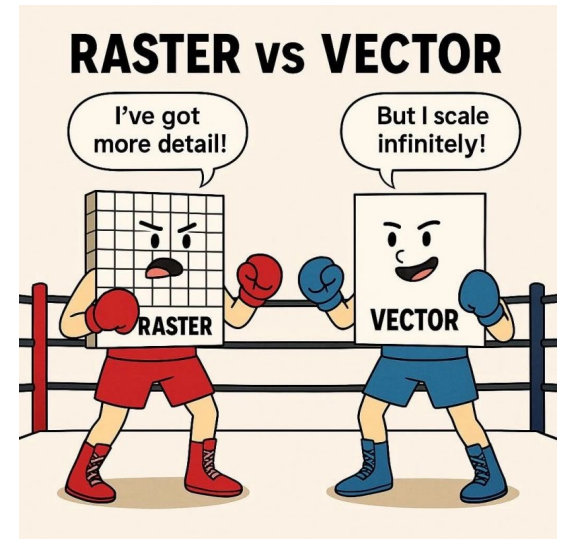
DAEN 489 ISEN 489 Spring 2026



ENGINEERING
TEXAS A&M UNIVERSITY

Overview

- Raster data represent continuous spatial variation as a grid of cells (pixels).
- Vector data represent discrete geographic features as points, lines, or polygons.
- Many practical geospatial problems require blending these models:
 - Satellite imagery + Administrative boundaries
 - Terrain surfaces + Sampled field locations
 - Continuous environmental surfaces + Spatial regions of interest
- The focus of raster–vector interaction is on how to **combine, extract, convert, and analyze together** raster and vector datasets in a consistent spatial framework.



Raster Cropping and Masking with Vector Boundaries

- Raster datasets often extend far beyond the area of interest.
- Cropping reduces spatial extent, improving performance.
- Masking sets values outside a vector boundary to **NA**, focusing analysis on relevant regions.
 - **crop()** reduces the raster's geographical extent.
 - **mask()** applies a spatial filter based on vector boundaries.
- We use both **crop()** and **mask()** together in most cases .
- This two-step process ensures efficient focus on the study area.

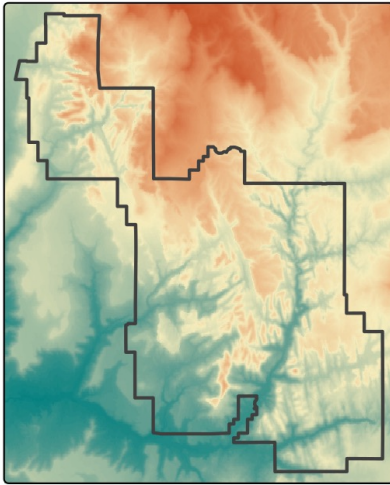
```
library(sf)
library(terra)
library(dplyr)

# Read and reproject data
srtm = rast(system.file("raster/srtm.tif", package
  = "spDataLarge"))
zion = read_sf(system.file("vector/zion.gpkg",
  package = "spDataLarge"))
zion = st_transform(zion, st_crs(srtm))

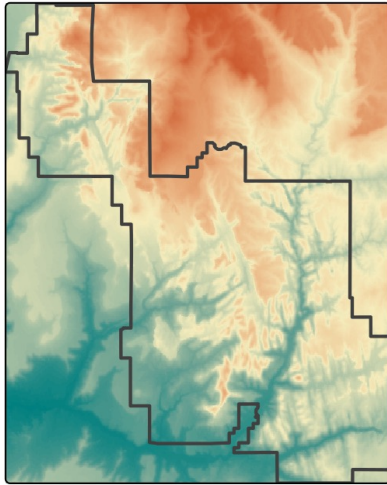
# Crop and mask
srtm_cropped = crop(srtm, zion)
srtm_masked = mask(srtm, zion)
srtm_cropped = crop(srtm, zion)
srtm_final = mask(srtm_cropped, zion)

srtm_inv_masked = mask(srtm, zion, inverse = TRUE)
```

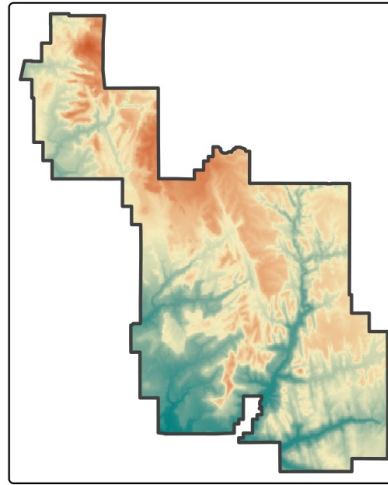
A. Original



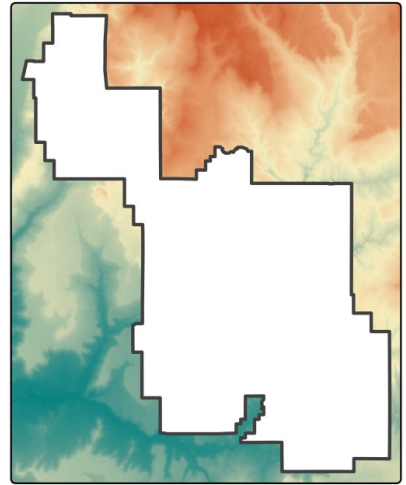
B. Crop



C. Mask



D. Inverse mask

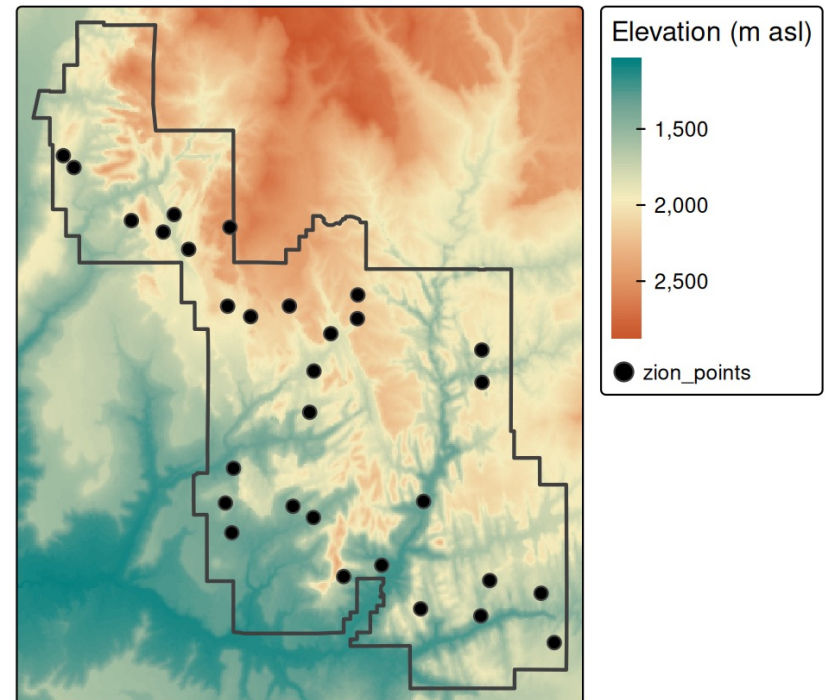


Raster cropping and raster masking.

Raster Extraction: Linking Raster Values to Vector Geometry

- Raster extraction retrieves raster values at spatial locations defined by a vector dataset.
- Depending on the vector feature type (points, lines or polygons) the nature of extracted results changes.
- Examples
 - **Points:** returns values at cell centroids closest to each point.
 - **Lines:** yields values for raster cells intersected by the line (or along sampled points).
 - **Polygons:** retrieves all cell values inside a polygon, allowing summary statistics.

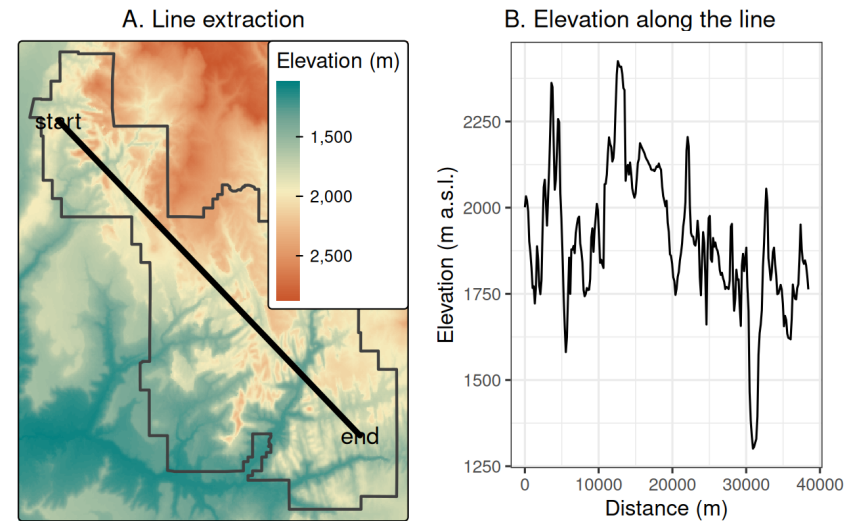
```
# Elevation Sampling (Points)  
data("zion_points", package = "spDataLarge")  
elevation = terra::extract(srtm, zion_points)  
zion_points = cbind(zion_points, elevation)
```



Locations of points used for raster extraction.

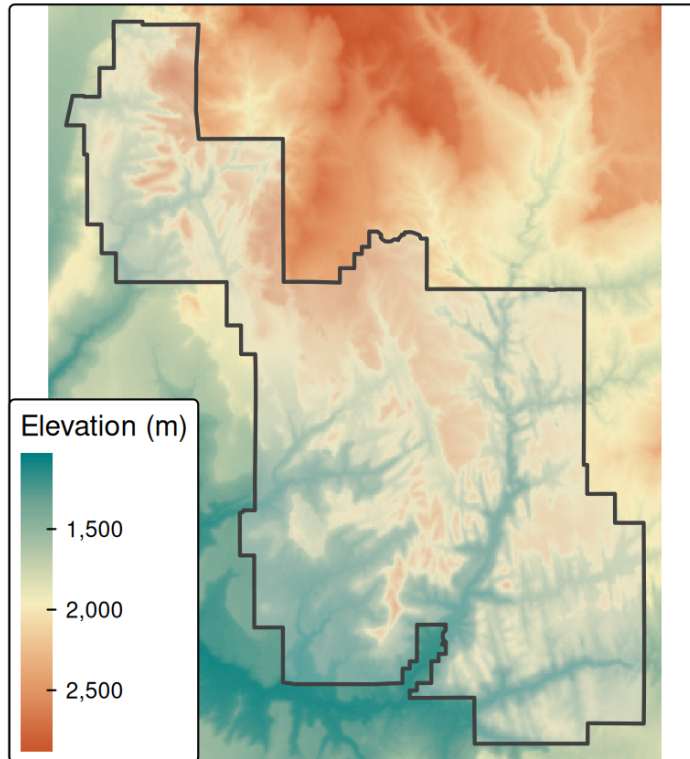
Raster Extraction With Line Selectors

```
zion_transect = cbind(c(-113.2, -112.9),  
  c(37.45, 37.2)) |>  
  st_linestring() |>  
  st_sfc(crs = crs(srtm)) |>  
  st_sf(geometry = _)  
  
zion_transect$id = 1:nrow(zion_transect)  
zion_transect = st_segmentize(zion_transect,  
  dfMaxLength = 250)  
zion_transect = st_cast(zion_transect,  
  "POINT")  
  
zion_transect = zion_transect |>  
  group_by(id) |>  
  mutate(dist = st_distance(geometry)[, 1])  
  
zion_elev = terra::extract(srtm,  
  zion_transect)  
  
zion_transect = cbind(zion_transect,  
  zion_elev)
```

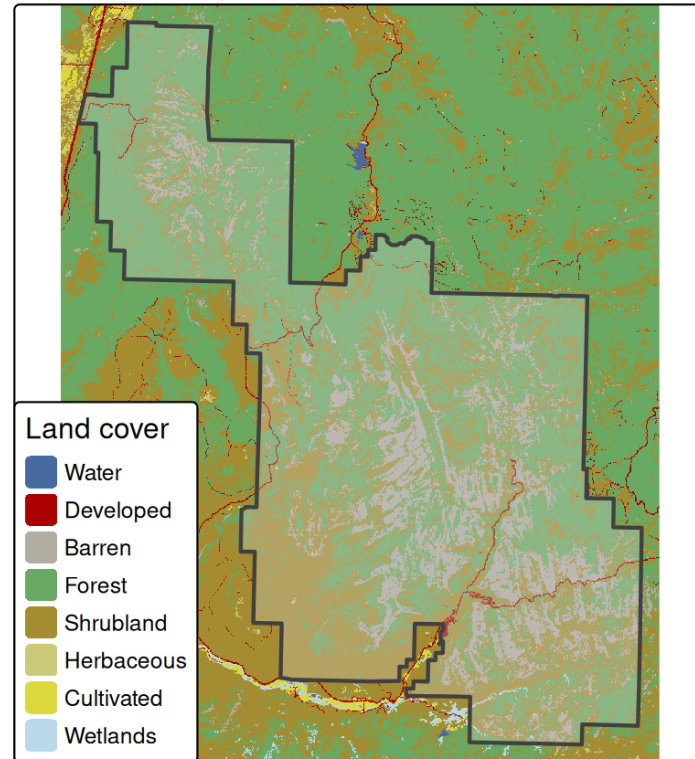


Location of a line used for (A) raster extraction and (B) the elevation along this line.

A. Continuous data extraction



B. Categorical data extraction



Area used for (A) continuous and (B) categorical raster extraction.

Rasterization: Converting Vector Features to Raster Form

- Rasterization transforms vector features into a raster grid aligned with a template:
 - Simplifies feature topology,
 - Enables raster-based modeling (e.g., machine learning, surface modeling),
 - Aligns disparate datasets to a common grid for analysis.
- Using varying summary statistics or criteria creates different raster outputs:
 - **Default:** presence/absence grid
 - **fun = "length"**
count of vector points per cell
 - **fun = sum on attribute**
aggregated attribute value per cell

```
cycle_hire_osm = spData::cycle_hire_osm
cycle_hire_osm_projected =
  st_transform(cycle_hire_osm, "EPSG:27700")

raster_template =
  rast(ext(cycle_hire_osm_projected),
  resolution = 1000, crs =
  crs(cycle_hire_osm_projected))

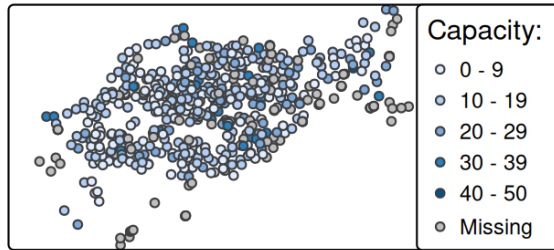
ch_raster1 =
  rasterize(cycle_hire_osm_projected,
  raster_template)

ch_raster2 =
  rasterize(cycle_hire_osm_projected,
  raster_template, fun = "length")

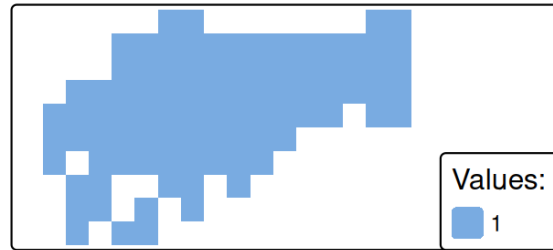
ch_raster3 =
  rasterize(cycle_hire_osm_projected,
  raster_template, field = "capacity", fun =
  sum, na.rm = TRUE)
```

Rasterization of lines and polygons requires careful handling of cell touch rules (e.g., using **touches = TRUE**).

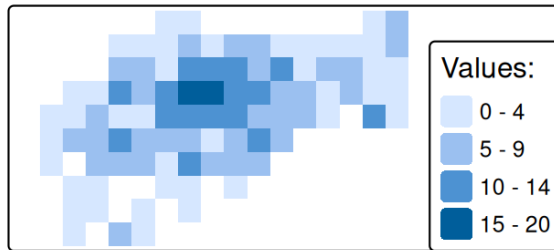
A. Points



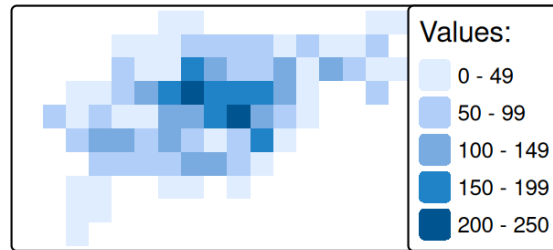
B. Presence/absence



C. Count



D. Aggregated capacity



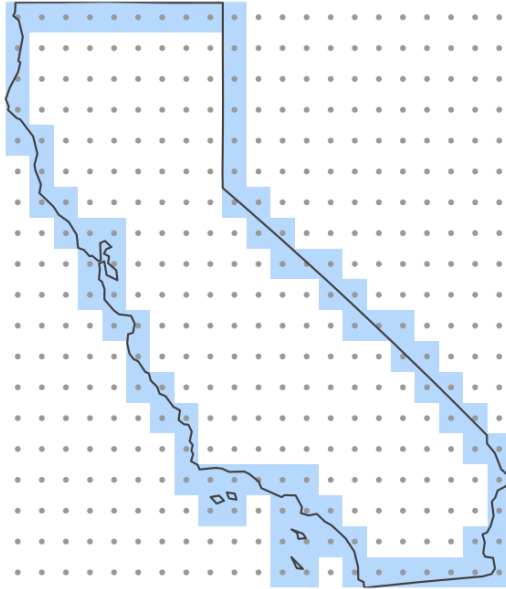
Examples of point rasterization.

Rasterization: Converting Vector Features to Raster Form

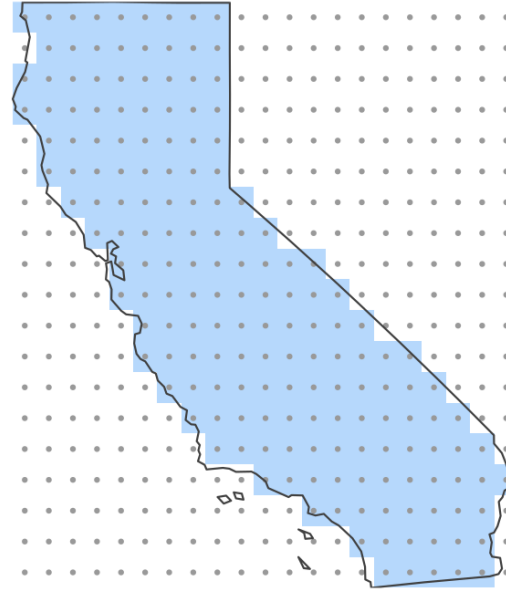
- Another dataset based on California's polygons and borders (created below) illustrates rasterization of lines.
- After casting the polygon objects into a multilinestring, a template raster is created with a resolution of a 0.5 degree.

```
california = dplyr::filter(us_states, NAME ==  
  "California")  
  
california_borders = st_cast(california,  
  "MULTILINESTRING")  
  
raster_template2 = rast(ext(california),  
  resolution = 0.5, crs =  
  st_crs(california)$wkt)  
  
california_raster1 =  
  rasterize(california_borders,  
    raster_template2, touches = TRUE)  
  
california_raster2 = rasterize(california,  
  raster_template2)
```

A. Line rasterization



B. Polygon rasterization



Examples of line and polygon rasterizations.

Spatial Vectorization: Raster → Vector Conversion

- Vectorized output may be better suited for feature extraction, topology building, or certain types of spatial queries.
- Common vectorization tasks include:
 - **Cell centroid extraction** → point representation
 - **Contour line generation** → isolines for continuous surfaces
 - **Polygonization** → vector polygons for raster classes or objects
- Vectorization supports advanced feature analysis and integration with other vector-based spatial models.

```
elev = rast(system.file("raster/elev.tif", package = "spData"))
elev_point = as.points(elev) |> st_as_sf()

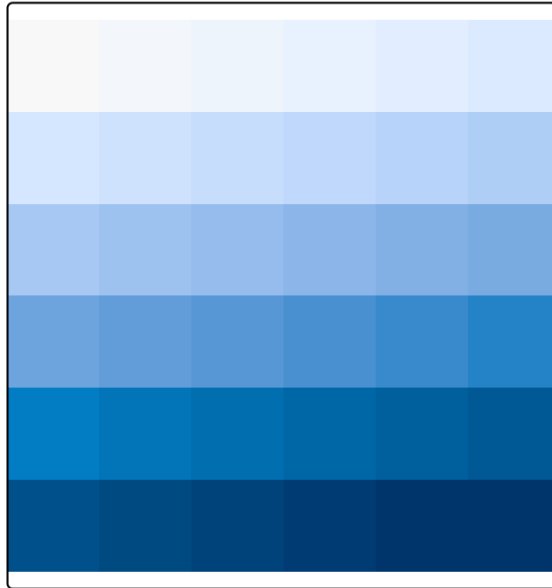
dem = rast(system.file("raster/dem.tif", package = "spDataLarge"))
cl = as.contour(dem) |> st_as_sf()
plot(dem, axes = FALSE); plot(cl, add = TRUE)

grain = rast(system.file("raster/grain.tif", package = "spData"))
grain_poly = as.polygons(grain) |> st_as_sf()
```

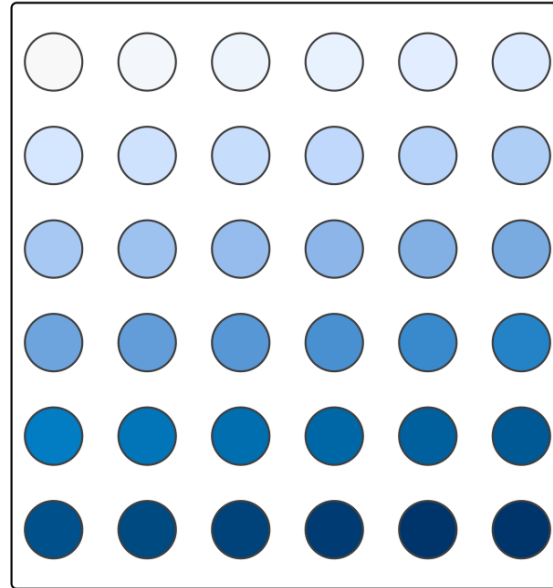
Examples:

- **Points:** every non-NA raster cell becomes a point geometry.
- **Contours:** lines representing constant value intervals.
- **Polygons:** convert clusters of cells into polygons, with optional aggregation via dissolved borders.

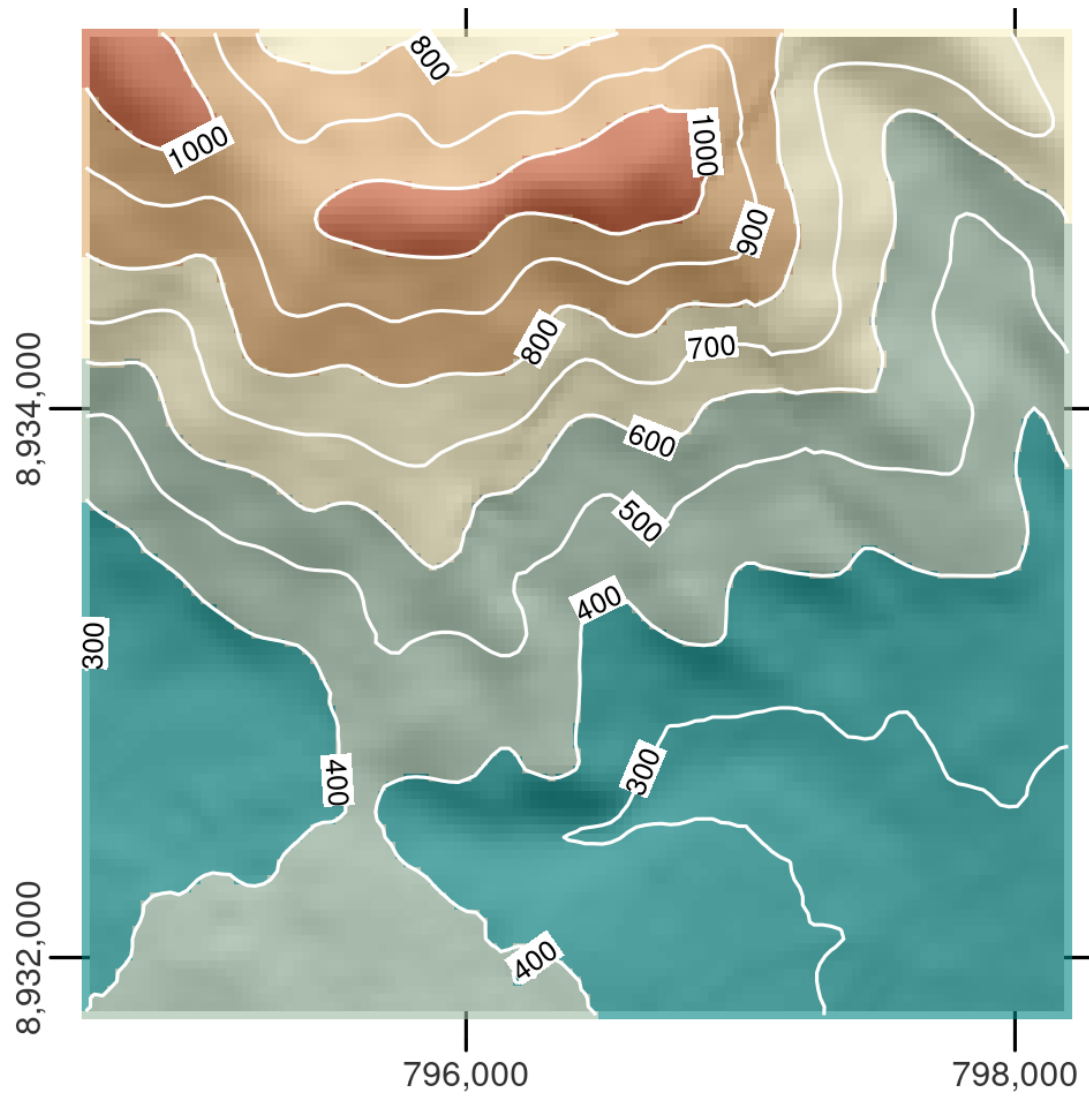
A. Raster



B. Points

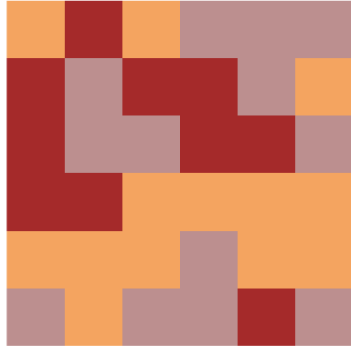


Raster and point representation of the elev object.

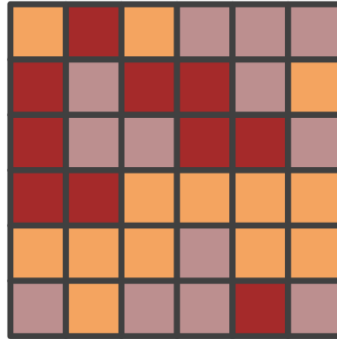


Digital elevation model with hillshading, showing the southern flank of Mt. Mongón overlaid with contour lines.

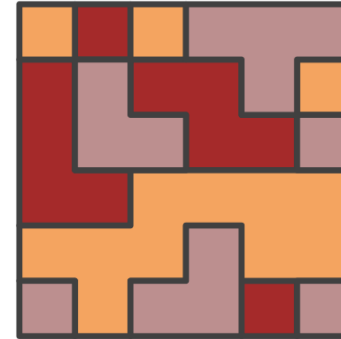
A. Raster



B. Polygons



C. Aggregated polygons



Vectorization of (A) raster into (B) polygons (**dissolve** = **FALSE**) and aggregated polygons (**dissolve** = **TRUE**).

Key Considerations

Integrating Raster & Vector Data

1. Coordinate Reference System (CRS) Consistency

- Always ensure that raster and vector data share the same CRS before interaction, otherwise extractions or overlays may misalign.

2. Resolution and Pixel Size Influence

- Raster resolution affects spatial detail and computational costs:
 - Too coarse → loss of meaningful detail.
 - Too fine → heavier processing and memory use.

3. Use Appropriate Extraction Tools

- Standard extraction tools may underperform on large datasets; specialized packages (e.g., [exactextractr](#)) offer faster and more precise operations for polygon extraction.

Practical Use (ideas for projects, maybe?)

1. Environmental & Civil Engineering

- Extract elevation values within watershed polygons.
- Rasterize stormwater network buffers to identify influence zones.
- Overlay rainfall rasters with parcel polygons for exposure assessment.

2. Transportation Engineering

- Extract raster values along road lines.
- Aggregate risk metrics by district polygon.
- Rasterize maintenance zones to align with predictive risk models.

3. Energy & Utilities Engineering

- Extract slope and vegetation cost surfaces along candidate corridors.
- Mask cost surfaces by exclusion zones.
- Rasterize corridor alternatives into a common cost grid.

4. Urban Systems Engineering

- Extract mean temperature by census tract.
- Polygonize high-temperature raster clusters.
- Join heat intensity with demographic indicators

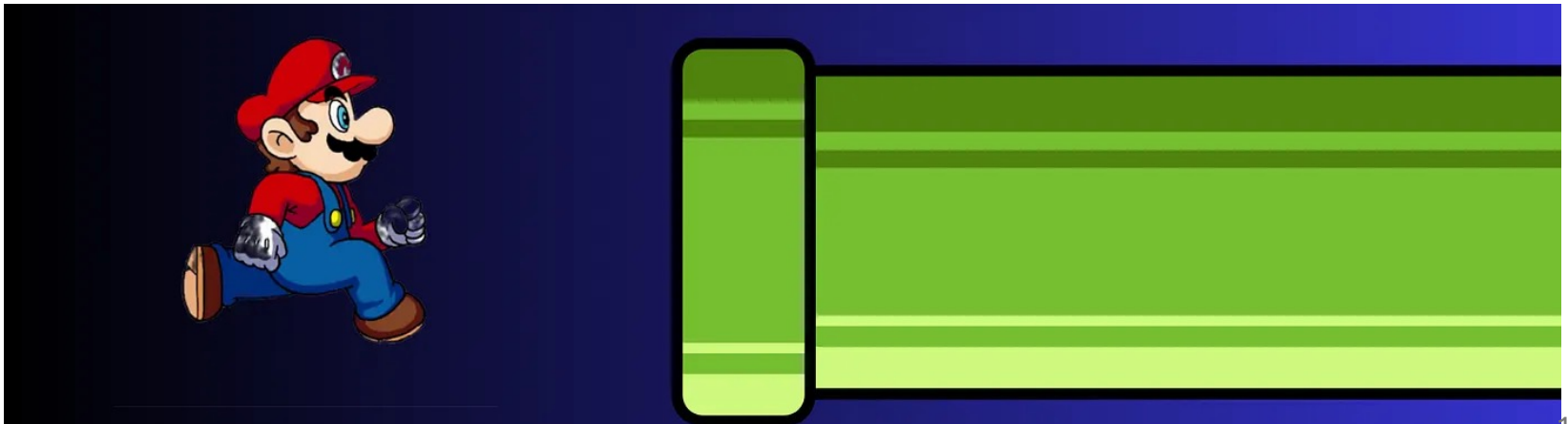
Raster-vector Workloads Are Compute-Intensive

- Common patterns:
 - Chunked raster processing
 - Parallel extraction jobs
 - Distributed tiling strategies
 - Serverless raster clipping APIs
- Engineering implications:
 - Partition by spatial tile.
 - Cache derived raster products.
 - Maintain metadata lineage for spatial transformations.



Data Pipelines & ETL Design

- Engineering teams can:
 - Store rasters in cloud object storage (Cloud-Optimized GeoTIFFs, COGs).
 - Store vectors in spatial databases (PostGIS).
 - Precompute extraction summaries for analytics layers.
 - Maintain consistent CRS and grid alignment standards.
- Rasterization is often used to:
 - Convert operational vector data into ML-ready gridded features.
 - Create feature stacks for spatial machine learning.



Data Pipelines & ETL Design

- Raster–vector interaction becomes a structured spatial ETL problem:

Stage	Example
Ingest	Load satellite raster + vector shapefiles
Transform	Reproject, align resolution, crop/mask
Enrich	Extract raster values to vector features
Aggregate	Summarize metrics per region
Serve	Store in spatial database or data warehouse

- Raster–vector interaction is not just GIS, it is:
 - Spatial feature engineering
 - Infrastructure modeling
 - Data-driven operations/intelligence
 - Scalable geospatial data systems

Practice Questions

1. *Suppose your Digital Elevation Model (DEM) has 30m resolution, but parcel boundaries are highly irregular and small. What biases might arise when extracting mean elevation per parcel?*
2. *How would raster resolution influence flood depth estimate when summarized at the parcel level?*
3. *If rainfall rasters and elevation rasters have different resolutions and CRSs, what preprocessing steps are required before interaction?*
4. *Design a reproducible pipeline that updates flood exposure metrics monthly as new rainfall rasters become available.*

